Funded by
the European Union

Workshop: ”Hands-on Kubernetes as a User”

The Presenter

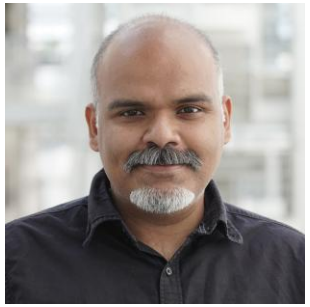
Daniel Medeiros

daniel.medeiros@ri.se

Researcher / MIMER and ENCCS

The Helpers (on chat)

- **Lodovico Giaretta**: Researcher at RISE / MIMER
- **Ashwin Mohanan**: Researcher at RISE / MIMER



General Guidelines

- Write your questions in the **HackMD** preferably (check link on chat), if they are not answered then we cover them during/after the break, or at the Q&A session at the end.
- Please mute yourself as we are a quite big number of people.
- You should have ideally installed k3d beforehand if you want to follow the hands-on part!
- **This workshop will be recorded!**
- Please be patient as live demos might have issues ☺

https://hackmd.io/@mimer-ai/hands-on-k8s/edit

Clone the repo! (link on chat)

https://github.com/mimer-ai/handson-k8s-workshop

Tentative Schedule

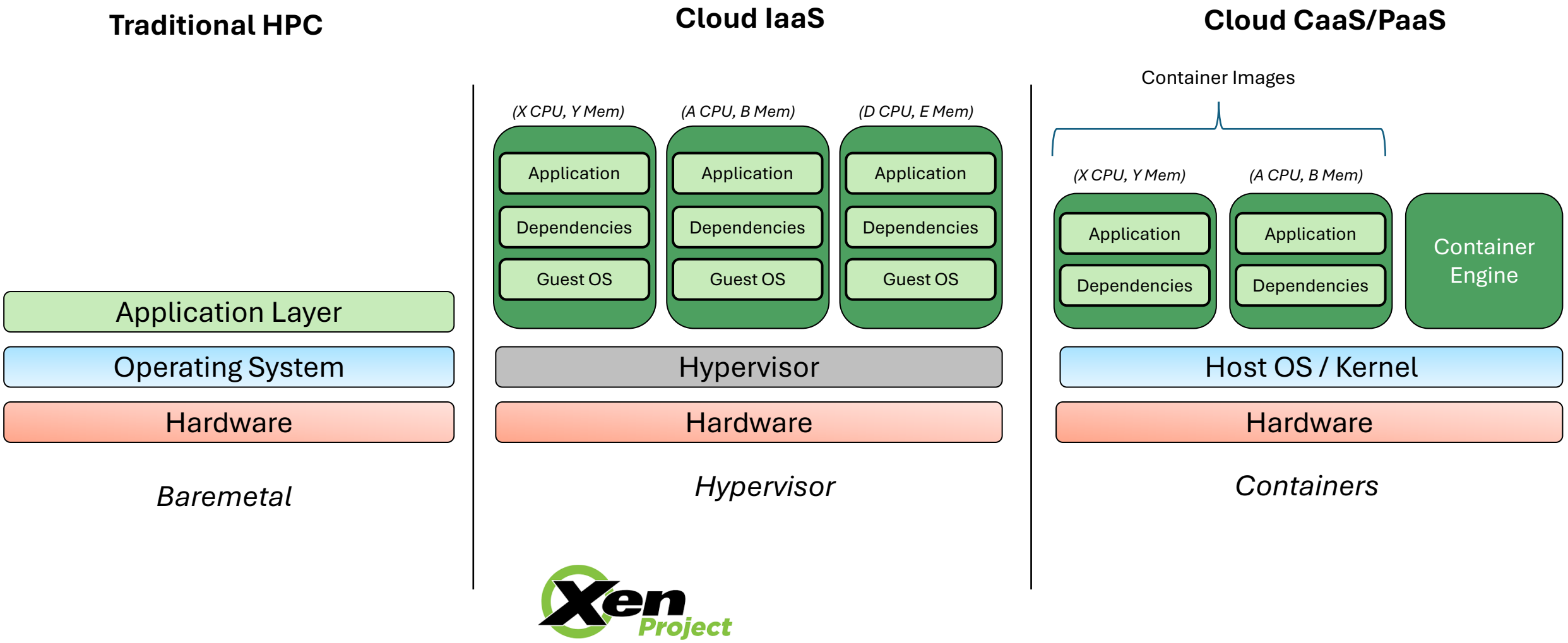
29 Apr 2026	What?
09:00 – 09:40	Theory: Introduction to Kubernetes
09:40 – 10:50	Theory 2: Concepts
10:50 – 11:00	Break
11:00 – 12:00	Hands-on 1: Jupyter, Prometheus, Grafana
12:00 – 13:15	Lunch
13:15 – 14:45	Hands-on 2: HPC and AI
14:45 – 15:00+	Q&A

What we are covering and not covering

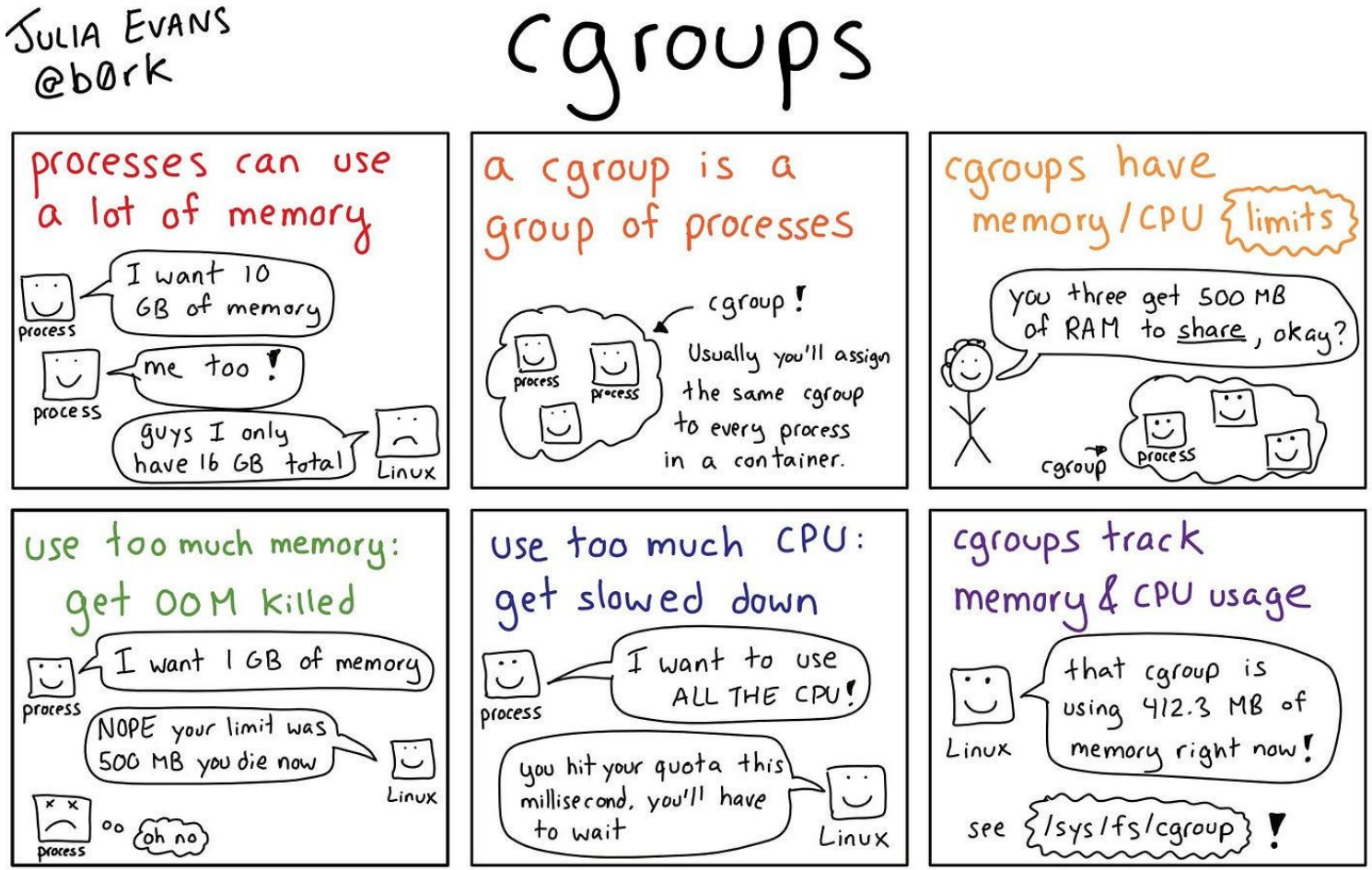
- YES: Using Kubernetes and its user interfaces
- YES: Understanding what are Kubernetes objects and how they work
- YES: Deploy applications and do some troubleshooting
- NO: Administering or build a production cluster (HA or non-HA)
- NO: Using specific APIs, building Operators or Device Plugins.

We start now ☺

Comparing different cloud models



Cgroups



We leverage *control groups* on Linux, while on other OS (Windows, MacOS) we often use a full VM (e.g. Moby Linux).

Container engines

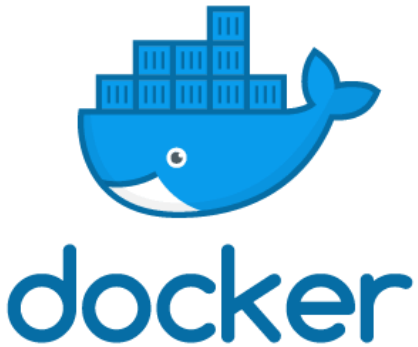
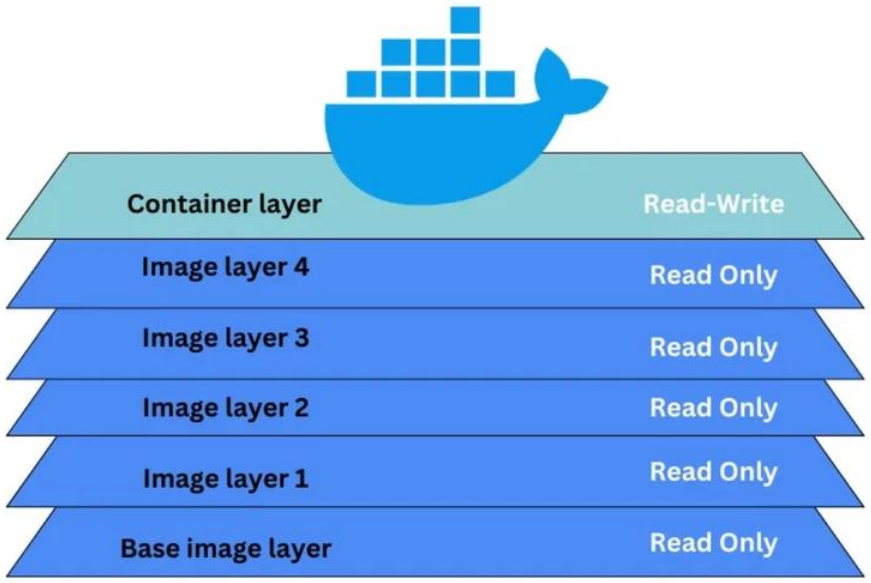
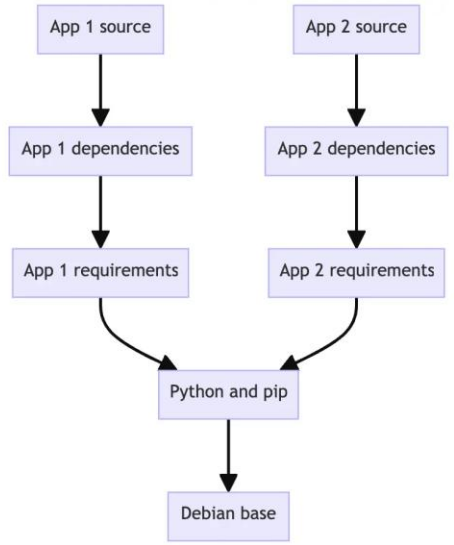


Image sources: project logos.

Container Engines – Docker layers



Layering structure



Reusing a layer for multiple images

Image sources: docker.io, dev.to.

Container Images

```
FROM debian:12-slim
RUN  apt-get update --fix-missing \
     && apt-get install -y git wget cmake zip unzip \
     && apt-get install -y build-essential gfortran libopenmpi-dev
\
     && apt-get install -y ssh time python3
RUN cd /home/ && wget ftp://ftp.gromacs.org/gromacs/gromacs-
2023.tar.gz \
    && tar xvf gromacs-2023.tar.gz && cd gromacs-2023 \
    && mkdir build && cd build \
    && cmake .. -DGMX_BUILD_OWN_FFTW=ON -
DREGRESSIONTEST_DOWNLOAD=ON -DGMX_MPI=ON && make && make install
WORKDIR /home
SHELL ["/bin/bash", "-l", "-c"]
RUN cat /usr/local/gromacs/bin/GMXRC.bash > /root/.bashrc
RUN mkdir -p /var/run/sshd;
CMD /usr/sbin/sshd;
```

Base image

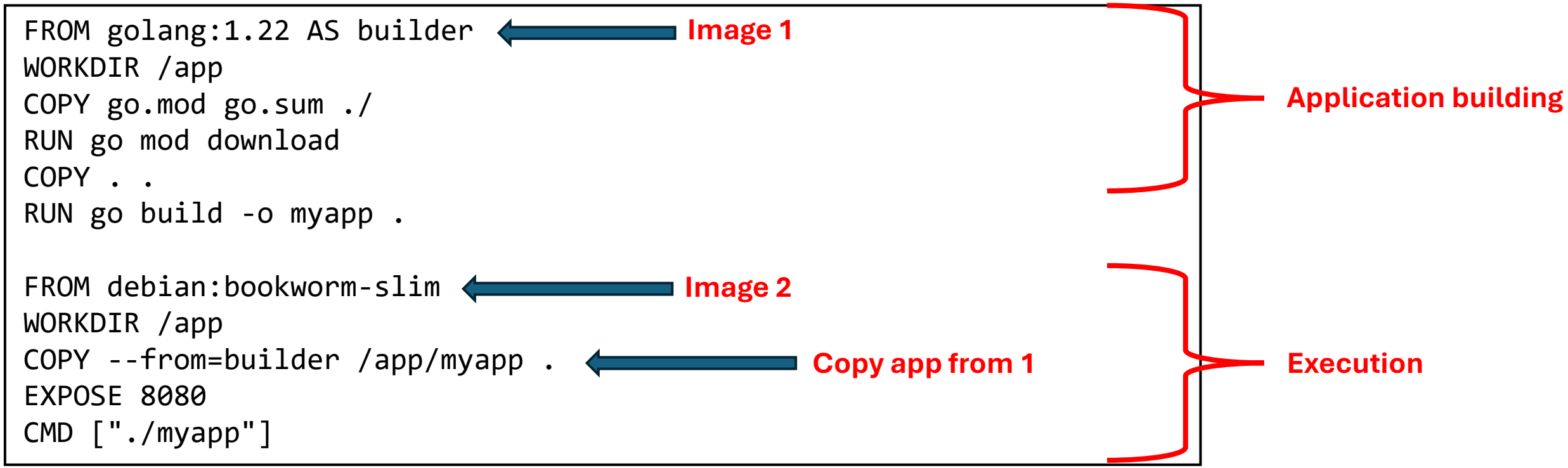
Dependencies

Application building

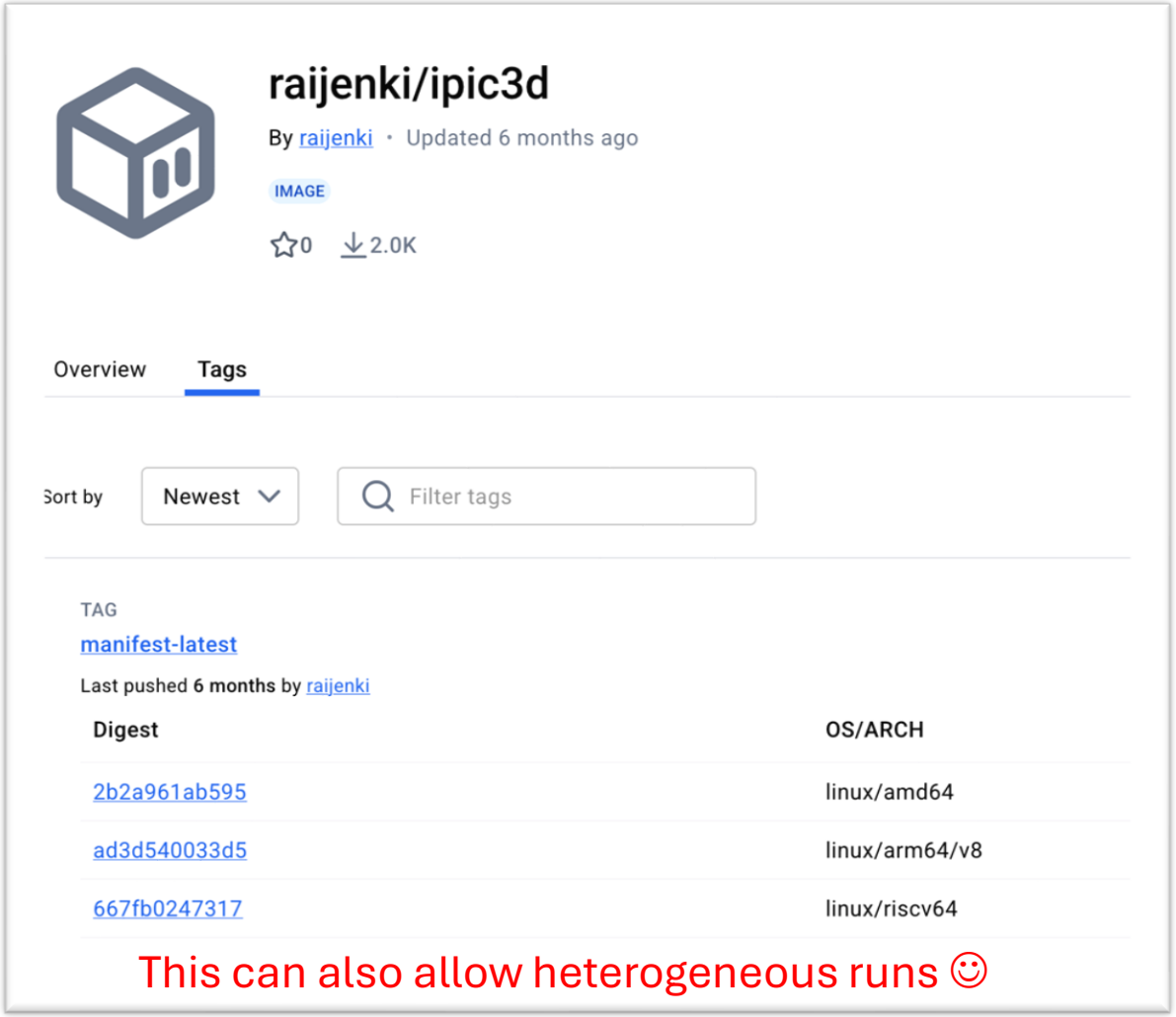
Post-build settings

Two-Stage Build

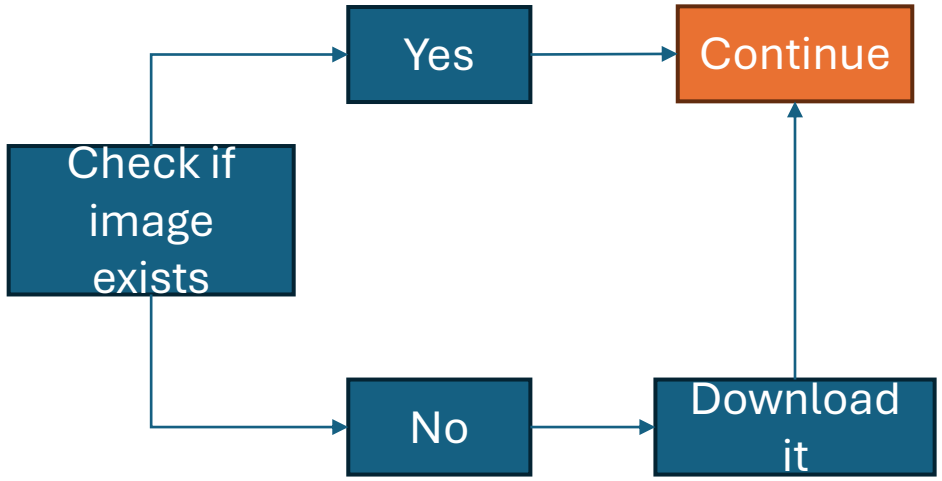
```
FROM golang:1.22 AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o myapp .

FROM debian:bookworm-slim
WORKDIR /app
COPY --from=builder /app/myapp .
EXPOSE 8080
CMD ["./myapp"]
```



Architecture and Repository



This can also allow heterogeneous runs ☺



Docker Execution Model

- Designed to be ephemeral, including storage;
- PID 1 in the container is the application layer that is supposed to be executed;
- Container lives while PID 1 lives.
- Images are pulled from a registry (e.g. dockerhub).

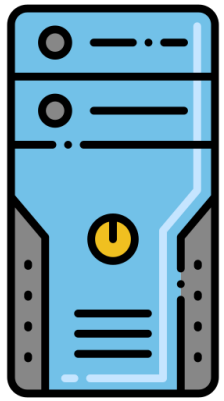
User request

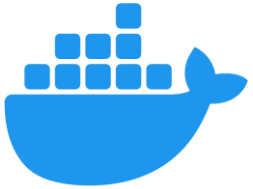
Docker daemon

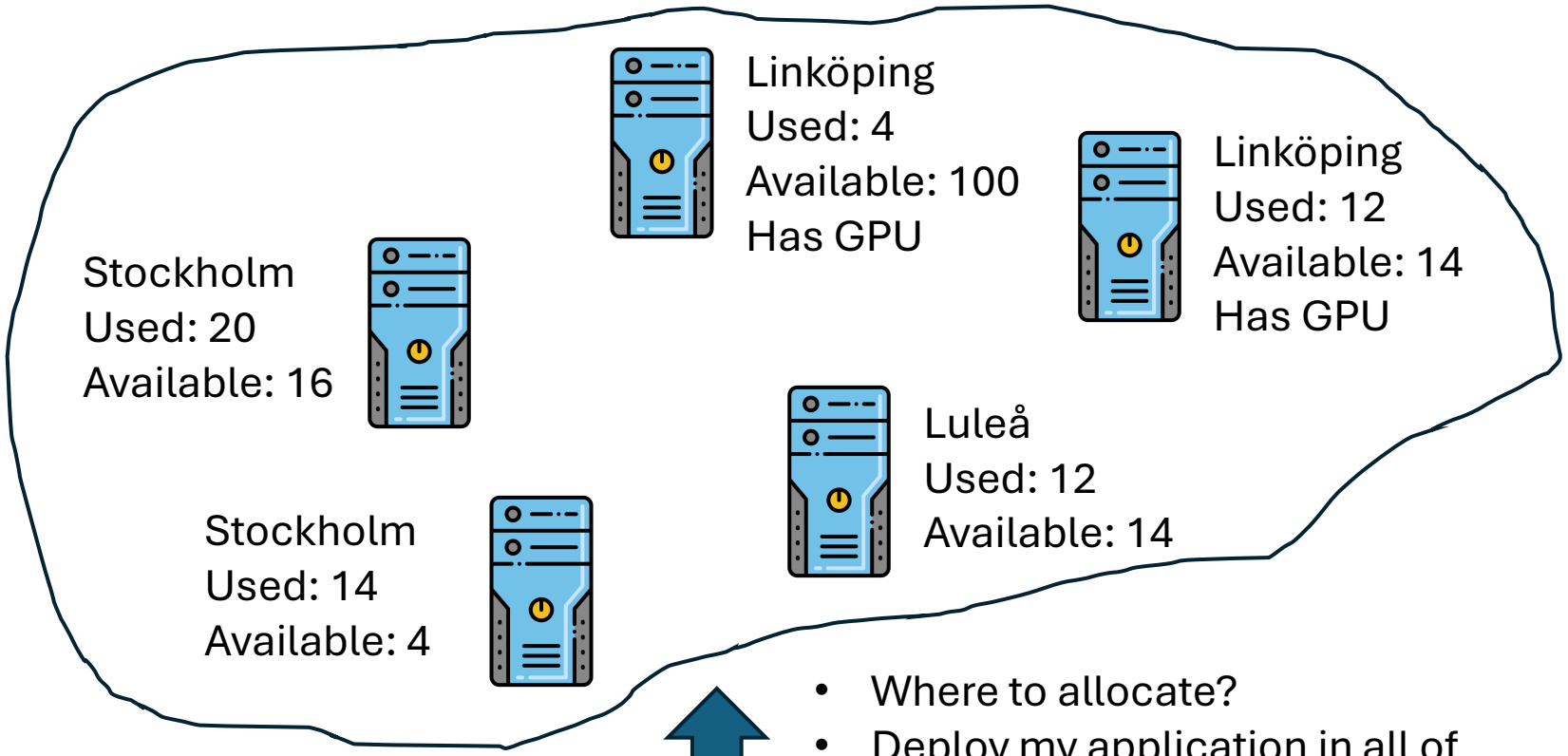
Linux kernel

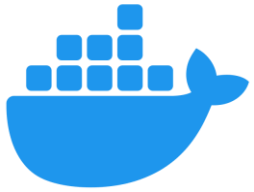
But this is only for one node so far…

What should I use Kubernetes for?









- Where to allocate?
- Deploy my application in all of them
- Scale my application as needed

(Some) Kubernetes Flavors

minikube

kubeadm

K3S

K3D

kubectl

```
❯ kubectl get pods
NAME                     READY   STATUS              RESTARTS   AGE
nginx-7c5ddbdf54-9d575   0/1     ContainerCreating   0          15m
nginx-7c5ddbdf54-f6wft   1/1     Running             0          15m
nginx-7c5ddbdf54-h6dnn   1/1     Terminating         0          15m
nginx-7c5ddbdf54-vtsqw   0/1     CrashLoopBackOff    0          15m

❯ kubectl apply -f resources.yaml
deployment.apps/my-deployment configured
service/my-service unchanged
configmap/my-config created
```

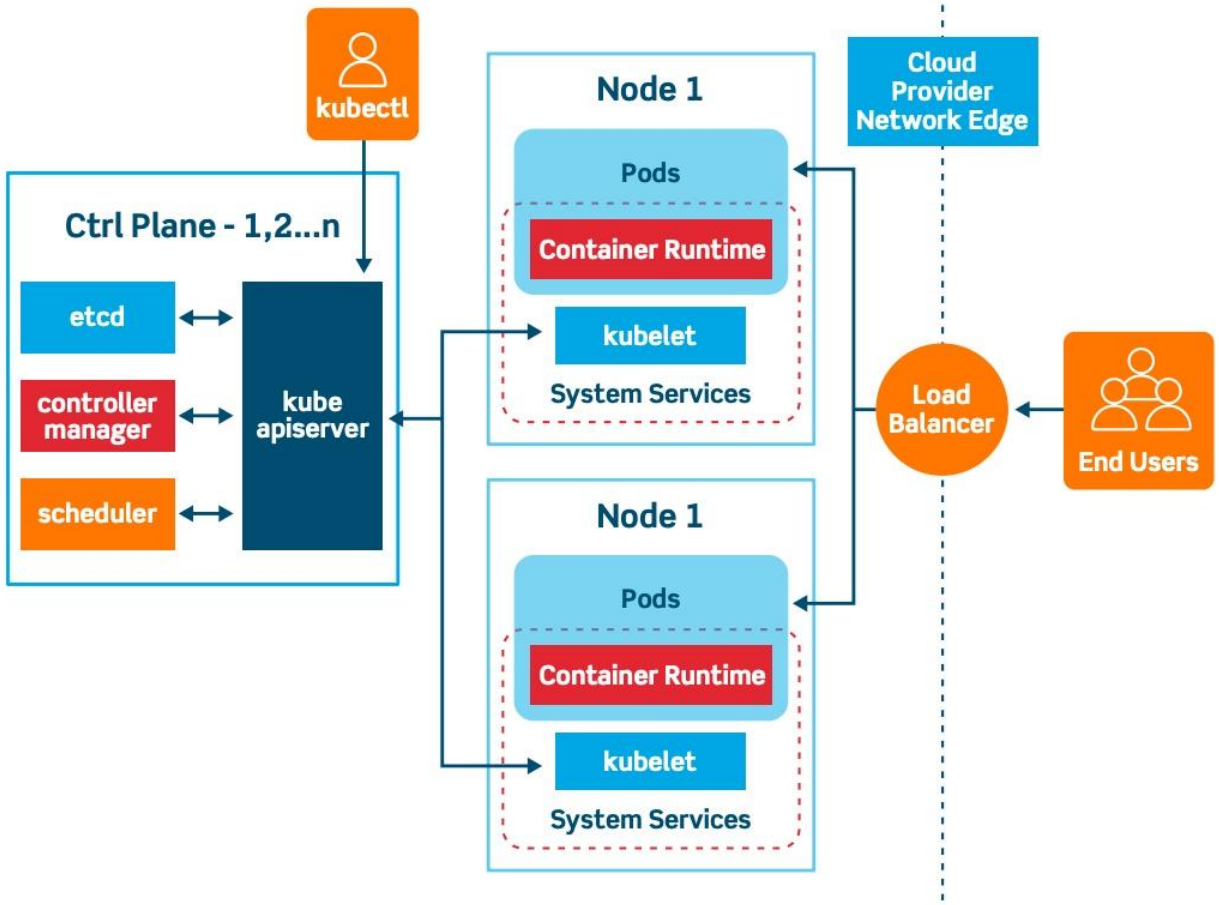
Also several clients: Python, Go, Java, C, others...

Another orchestrators:

HashiCorp Nomad

DOCKER SWARM

Architecture of a Kubernetes Cluster



Always need:

- api-server
- scheduler
- controller manager
- Etcd object storage
- Kubelet
- Container runtime

Addons:

- DNS
- Networking interface

Structure of a Kubernetes object

```
apiVersion:
kind:
metadata:

spec:
```

Structure of a Kubernetes object

```yaml
apiVersion: v1
kind: Pod
metadata:
        name: nginx
spec:
        containers:
                - name: nginx
                image: nginx:1.14.2
                ports:
                - containerPort: 80
```

- Pods are an abstraction of resources, made for long-running applications.
- Default pattern is to restart if fail.

Structure of a Kubernetes object

```
apiVersion: v1
kind: Pod
metadata:
        name: nginx
spec:
        containers:
                - name: nginx
                image: nginx:1.14.2
                ports:
                - containerPort: 80
        resources:
         requests:
           memory: “128Mi”
           cpu: “250m”
         limits:
           memory: “120Mi”
           cpu: ”500m”
```

- Pods are an abstraction of resources, made for long-running applications.
- Default pattern is to restart if fail.

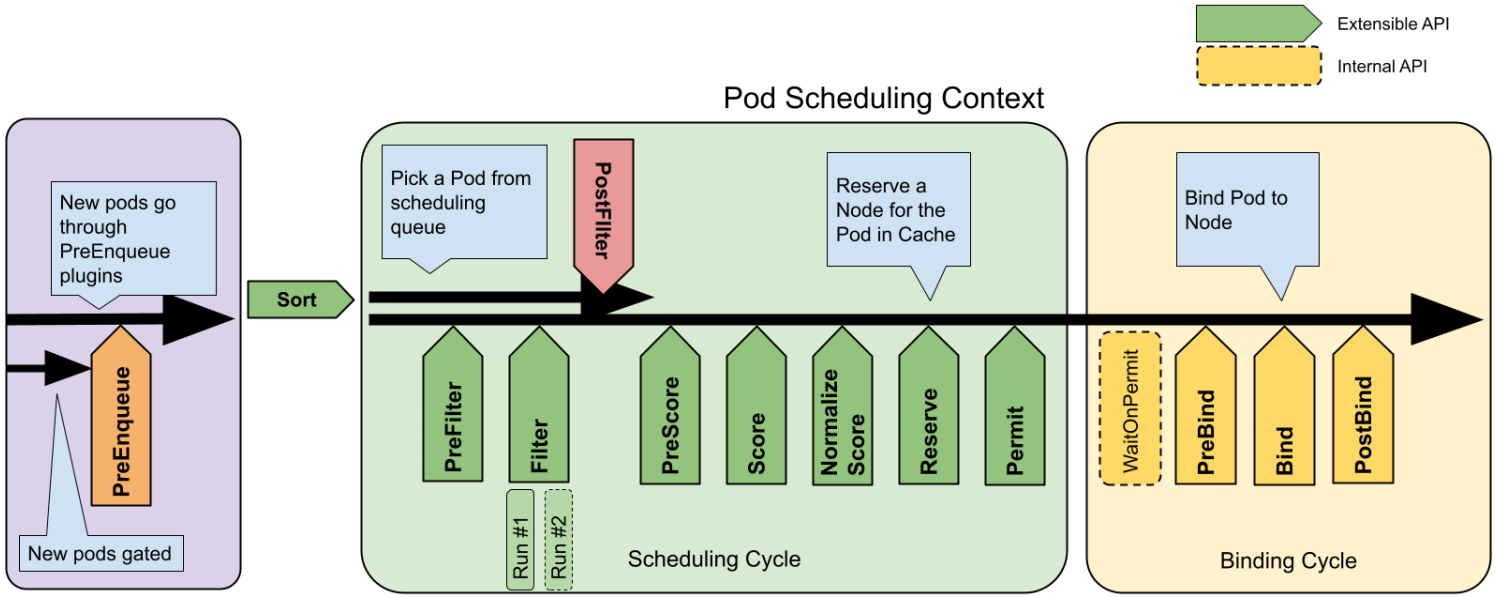
- Other relevant field is **imagePullPolicy**.

Can also be used for custom resources, GPUs, FPGAs, etc…

How the scheduler works?

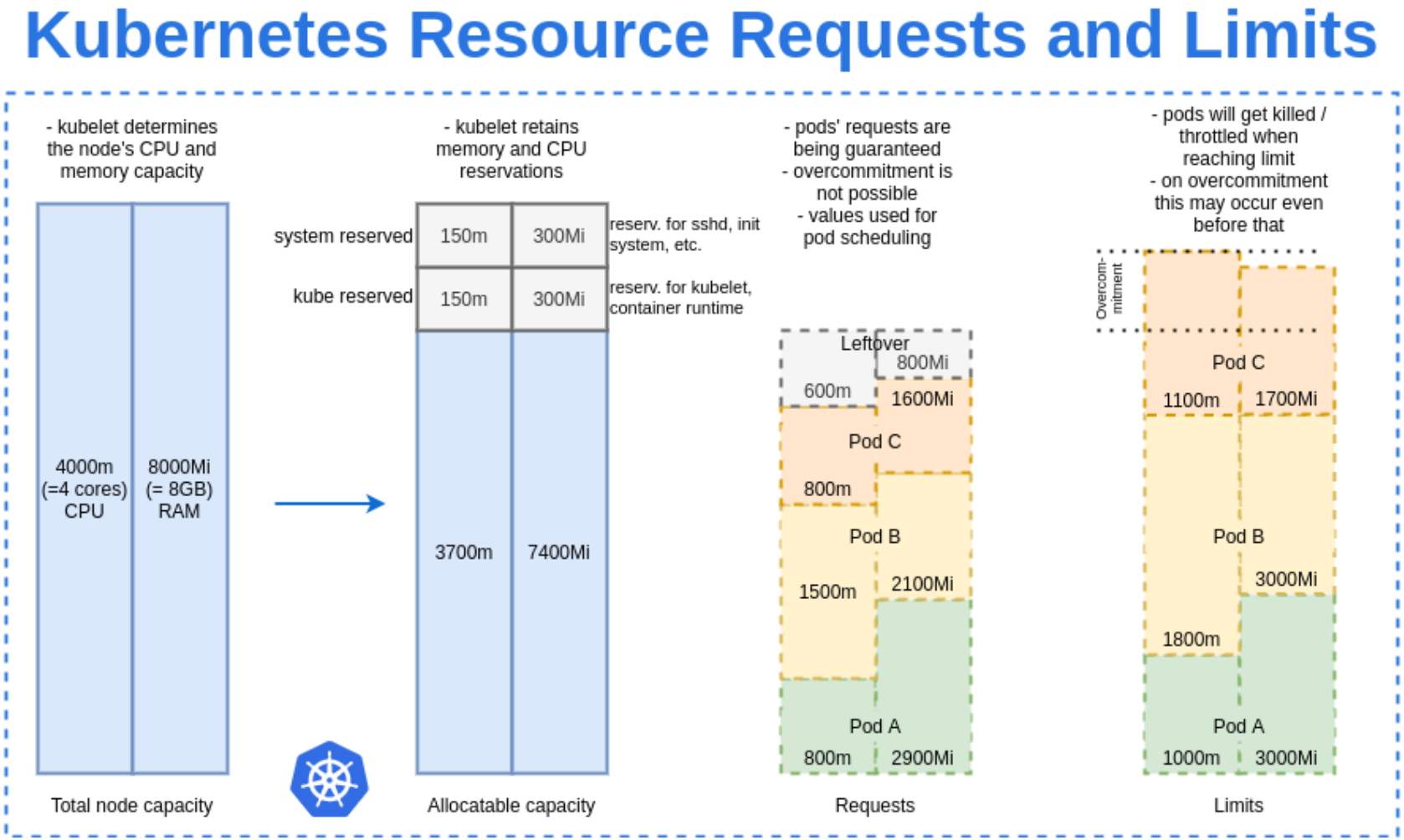
Two Phases:

1. **Scheduling**, where the scheduler filters (based on resources, taints and others) and scoring (uses a scoring function to determine the best node)
2. **Binding**: Putting the pod to the node with the highest score



Source: kubernetes.io

Requests vs Limits



Source: shipit.dev

Let’s start our cluster and use our first kubectl

Create cluster in k3d:
sudo k3d cluster create my-cluster

See active nodes and pods:
sudo kubectl get nodes
sudo kubectl get pods -A
sudo kubectl get pod <pod-name> -n <namespace>
sudo kubectl describe node <node-name>

Create, examine, and enter into a pod:
sudo kubectl create –f <pod_A.yaml> -f <pod_B.yaml>
sudo kubectl describe pod <pod_name>
sudo kubectl exec –it <pod_name> -- /bin/bash

Delete a pod:
sudo kubectl delete –f <pod_A.yaml> -f <pod_B.yaml>
OR
sudo kubectl delete pod <pod-name>

Other interesting commands:

- kubectl explain: see help
- kubectl edit: edit existing object

	Kubectl create	Kubectl apply	Kubectl replace
Resource doesn't exist	Creates it	Creates it	Fails
Resource already exists	Fails	Update it	Replaces it
Partial update	No	Only changed fields	Full replacement
Deletes and recreates	No	No	Yes
Idempotent (run several times, get same result)	No	Yes	Yes


DEMO
TIME

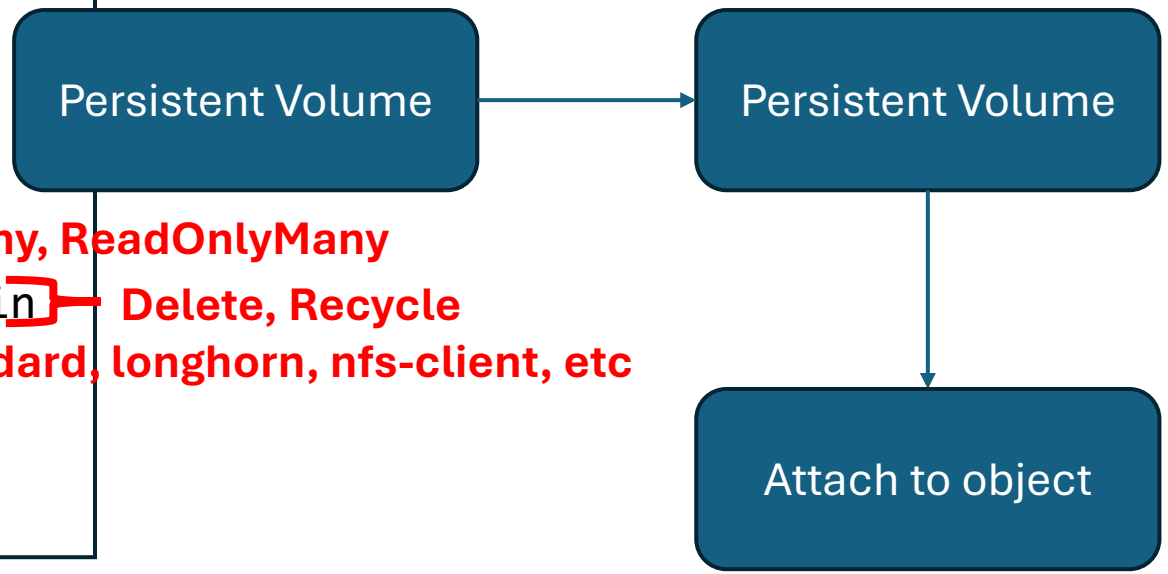
Attaching volumes in a pod

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: busybox-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-path
  hostPath:
    path: /your/path/here
```

ReadWriteMany, ReadOnlyMany

Delete, Recycle

Standard, longhorn, nfs-client, etc

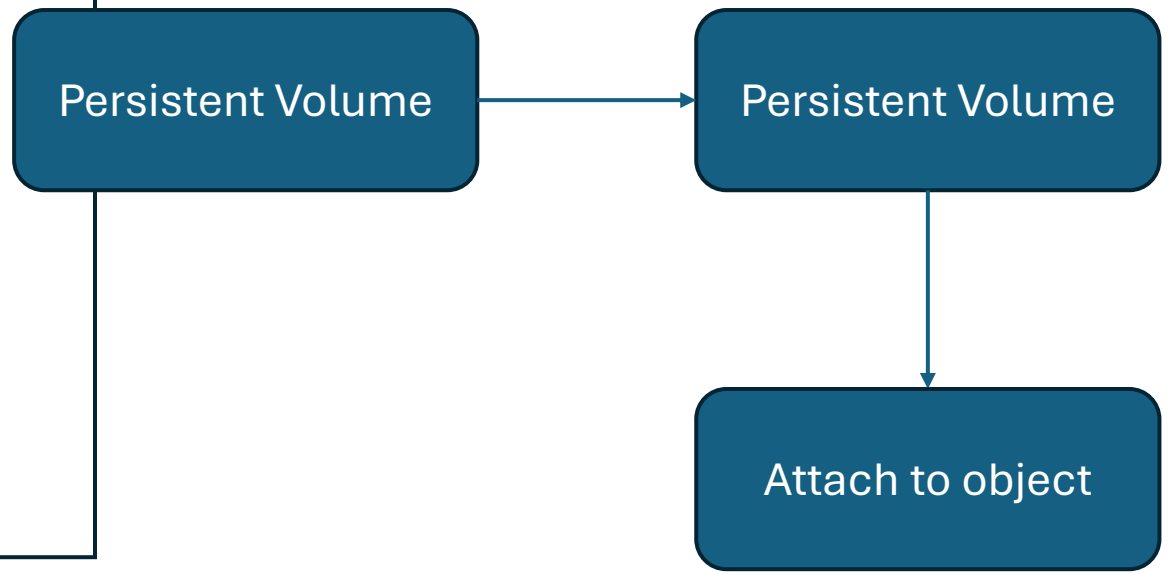


Since we are in k3d, we need to create the cluster with –volume flag

sudo k3d cluster create my-cluster --volume /home/daniel/handson-k8s-workshop/volumes:/pv-data@all

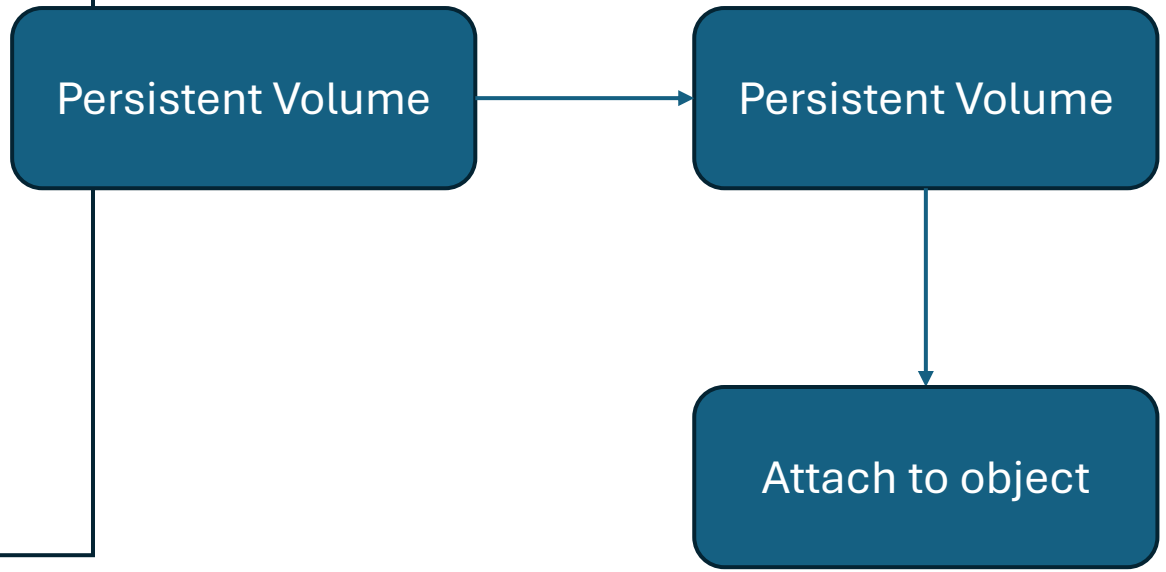
Attaching volumes in a pod

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: busybox-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
  resources:
    requests:
      storage: 1Gi
```



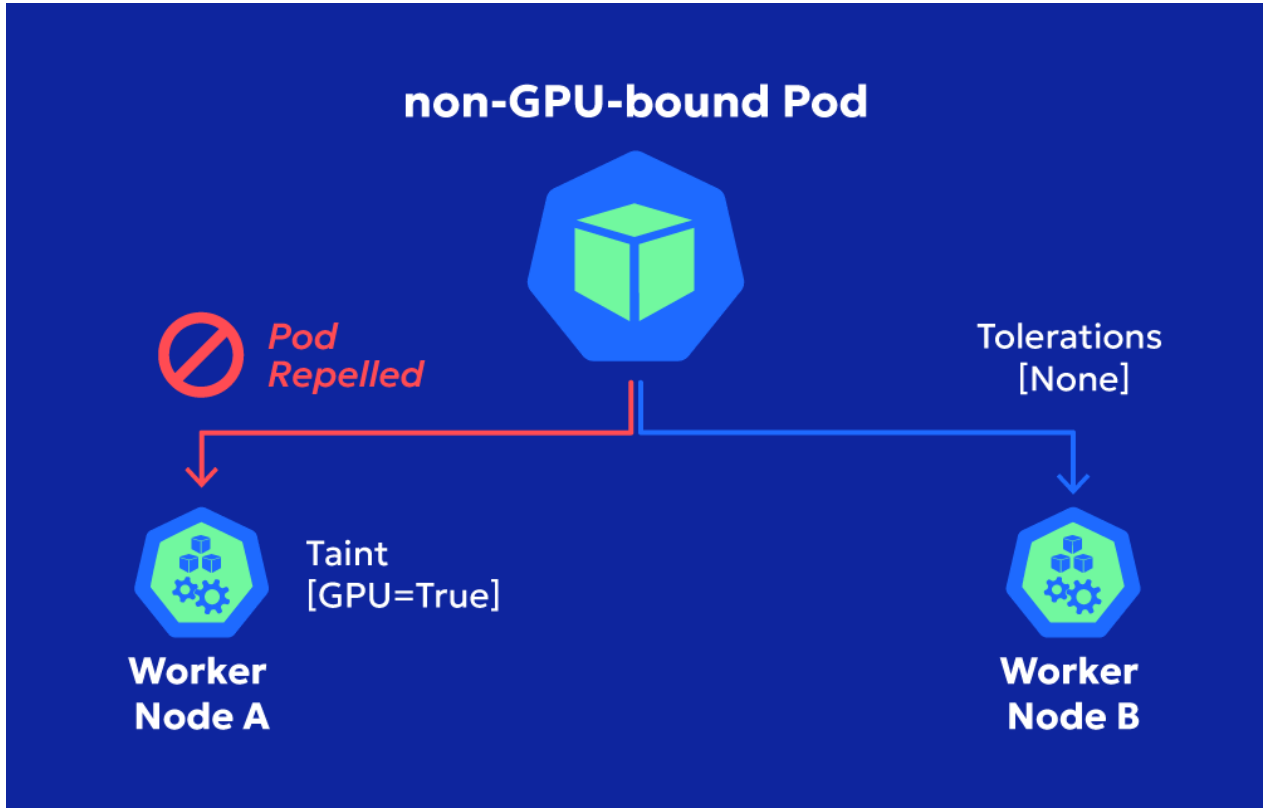
Attaching volumes in a pod

```
containers: .......
  volumeMounts:
      - name: persistent-storage
        mountPath: /data
volumes:
  - name: persistent-storage
    persistentVolumeClaim:
      claimName: busybox-pvc
```




DEMO
TIME

Taints and Tolerations



Source: Zesty.co

Let’s test the taints and tolerations!

Create a new node in k3d:

sudo k3d node create worker -c <cluster-name>

Label the node (optional):

sudo kubectl label node <node-name> node-role.kubernetes.io/worker=

Taint the node:

sudo kubectl taint node <node-name> key1=value1:NoSchedule

Add the toleration

```
containers: .......
tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoSchedule"
```

Exist

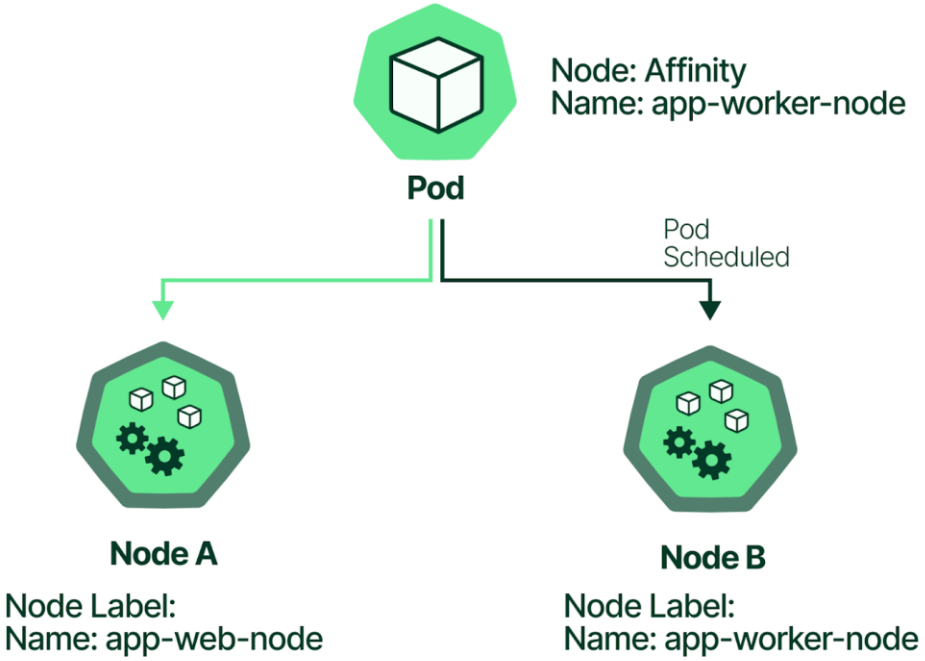
NoExecute

Let’s test the taints and tolerations!

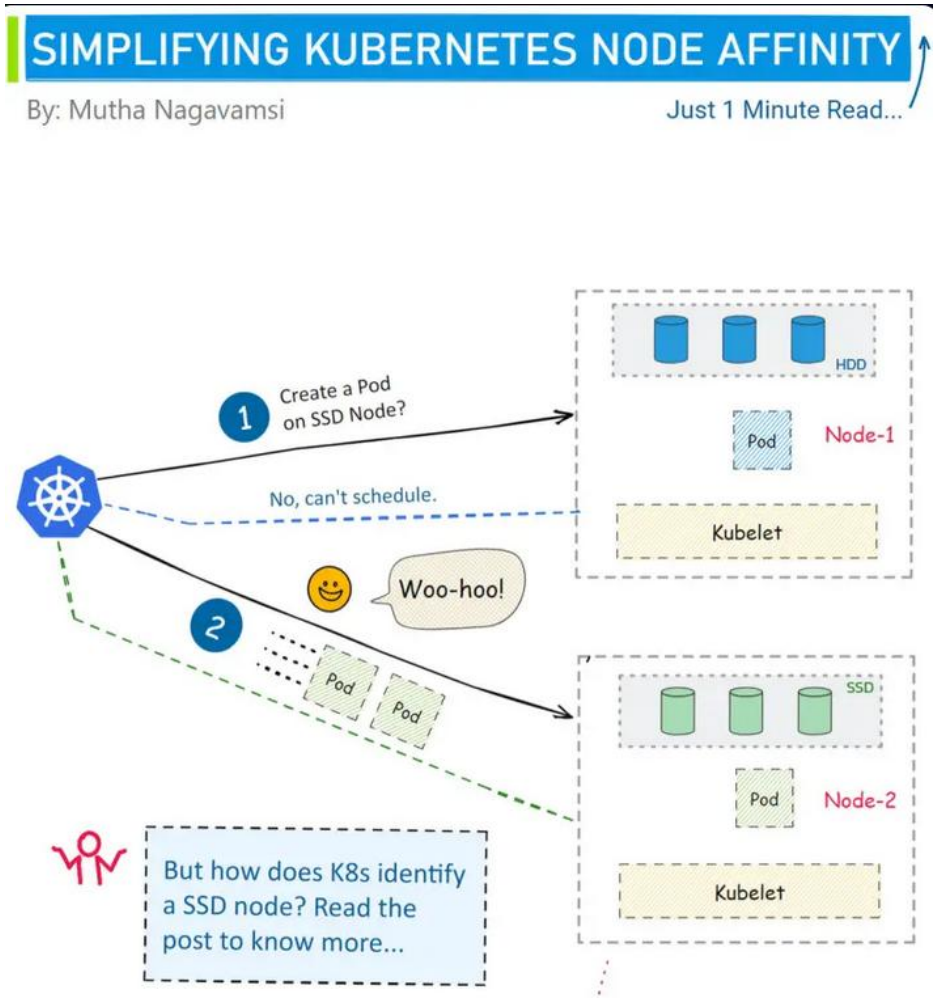
Delete the taint:

sudo kubectl taint node <node-name> key1=value1-

Node Affinity



Source: Apptio



Source: Linuxhandbook.com

NodeSelector

Add a label:
sudo kubectl label node k3d-worker-0 gpu=true

```
spec:
  nodeSelector:
    gpu: "true"
  containers:
    - name: alpine
      image: alpine
      command: ["sleep", "3600"]
```

Only “Equal” operator

Remove a label:
sudo kubectl label node k3d-worker-0 gpu-

NodeAffinity

Add a label:
sudo kubectl label node k3d-worker-0 gpu=true

```
containers: .......
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
       - matchExpressions:
         - key: gpu
           operator: In
           values:
           - “true”
```

Required vs preferred: hard vs soft rule

node-role.kubernetes.io/worker

Exists

Remove a label:
sudo kubectl label node k3d-worker-0 gpu-


DEMO
TIME

ReplicaSet

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: nginx-rs
spec:
 replicas: 3
 selector:
  matchLabels:
   app: nginx
 template:
  metadata:
   labels:
    app: nginx
  spec:
   containers:
    - name: nginx
     image: nginx
```

Pretty much a pod

- Keeps the number of pods all the time

DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: nginx-ds
spec:
 selector:
  matchLabels:
   app: nginx
 template:
  metadata:
   labels:
    app: nginx
  spec:
   containers:
    - name: nginx
      image: nginx
```

Pretty much a pod

- One pod per node! (that's why no number of replicas here)
- Scales with the cluster nodes (one new node = one new replica)
- Allows you to do rolling updates!

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deploy
spec:
 replicas: 3
 selector:
  matchLabels:
   app: nginx
 template:
  metadata:
   labels:
    app: nginx
  spec:
   containers:
    - name: nginx
     image: nginx
```

- Extends ReplicaSet!
- Allows you to do Rolling Updates, and Rollback.

These are for stateless applications, where each pod is equal!

Namespaces

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```

- In multi-tenant clusters, you are often restricted to your own namespace.

- Alternatively:

kubectl create namespace my-namespace

Execute pods with the –n flag if necessary:
Kubectl create –f <file.yaml> -n my-namespace

Leave your namespace as default:
kubectl config set-context --current --namespace=my-namespace

Jobs

- Designed for finite-running tasks
- Relevant attributes:
  - **backOffLimit**: How many times to retry after failure
  - **Completions**: How many times it should succeed
  - **Parallelism**: How many pods run in parallel
  - **activeDeadlineSeconds**: Kill the job after X seconds.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
 name: hello-job
spec:
 template:
  spec:
   restartPolicy: Never
   containers:
    - name: hello
      image: alpine
      command:
       - sh
       - -c
       - |
         echo "Job started at $(date)"
         echo "Running some work..."
         sleep 5
         echo "Computing something..."
         echo "1 + 1 = 2"
         echo "Job finished at $(date)"
 backoffLimit: 3
```

Let’s test the different apps!

Create more nodes in k3d:
sudo k3d node create worker -c <cluster-name>

Then just execute the yaml files to see the replicaset and daemonset!

Check logs with:
sudo kubectl logs <pod-name>

Let’s test rolling out and back for Deploy!

Create the deployment:
sudo kubectl create -f 10_Deployment_with_RollingUpdates.yaml

Change the yaml file with a new version of nginx (e.g. 1.26), and:
sudo kubectl apply –f 10_Deployment_with_RollingUpdates.yaml

Now roll back to previous version:
kubectl rollout history deployment/nginx-deploy
kubectl rollout undo deployment/nginx-deploy


DEMO
TIME

Kubernetes Networking Model

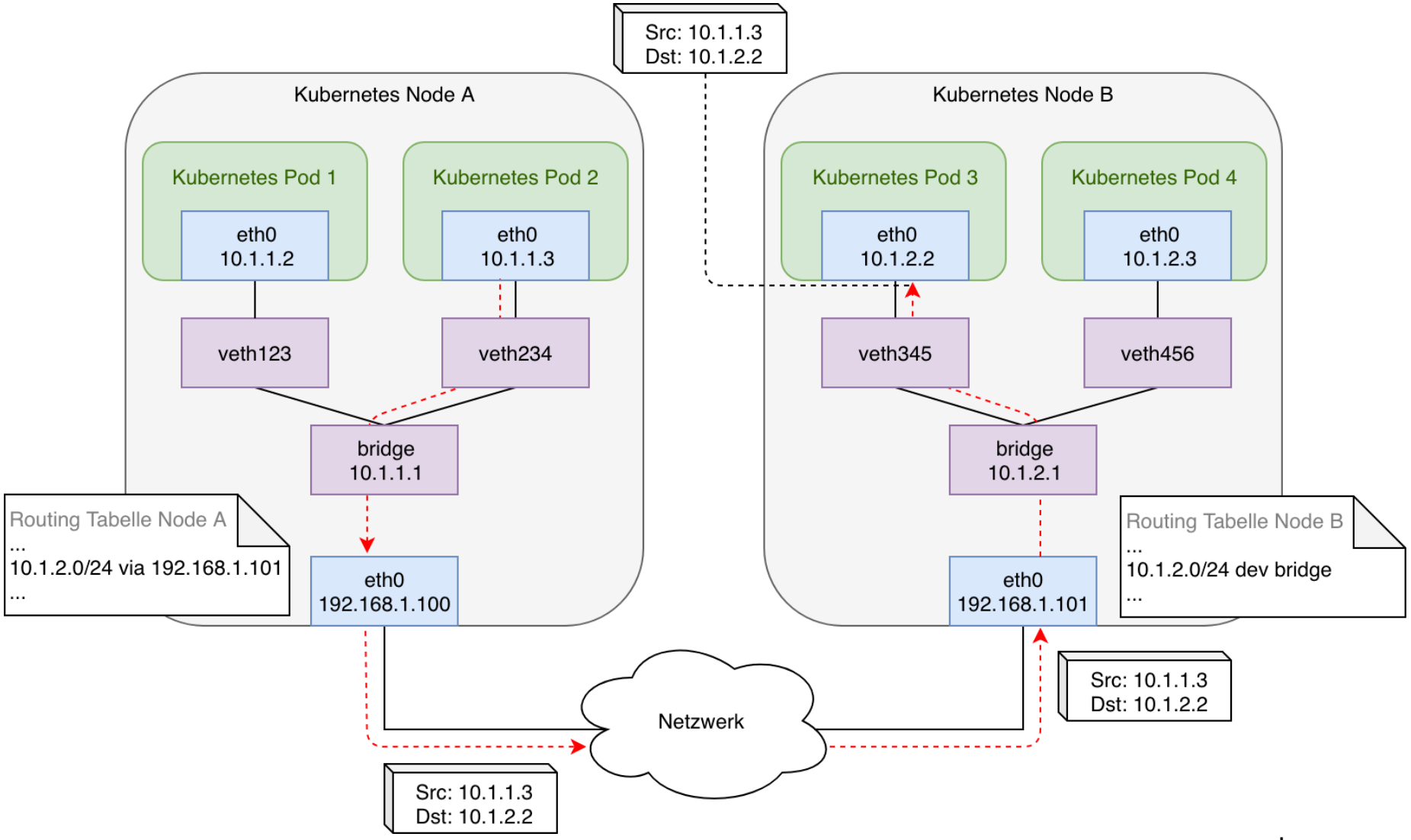


Image sources: inovex.de.

Kubernetes Networking Model - CNI

Calico

- For production environment
- Advanced network policy control
- Observability, encryption

flannel

- Simple setups
- Limited scalability
- No network policy support
- No encryption, no observability

That’s the one used defautly by k3d

Services – Why?

So A wants to talk with B.

Without Service:

Pod B IP = 10.0.0.2 —► Pod B crashes and restarts
Pod B IP = 10.0.0.8 —► Pod A is now talking to the wrong IP

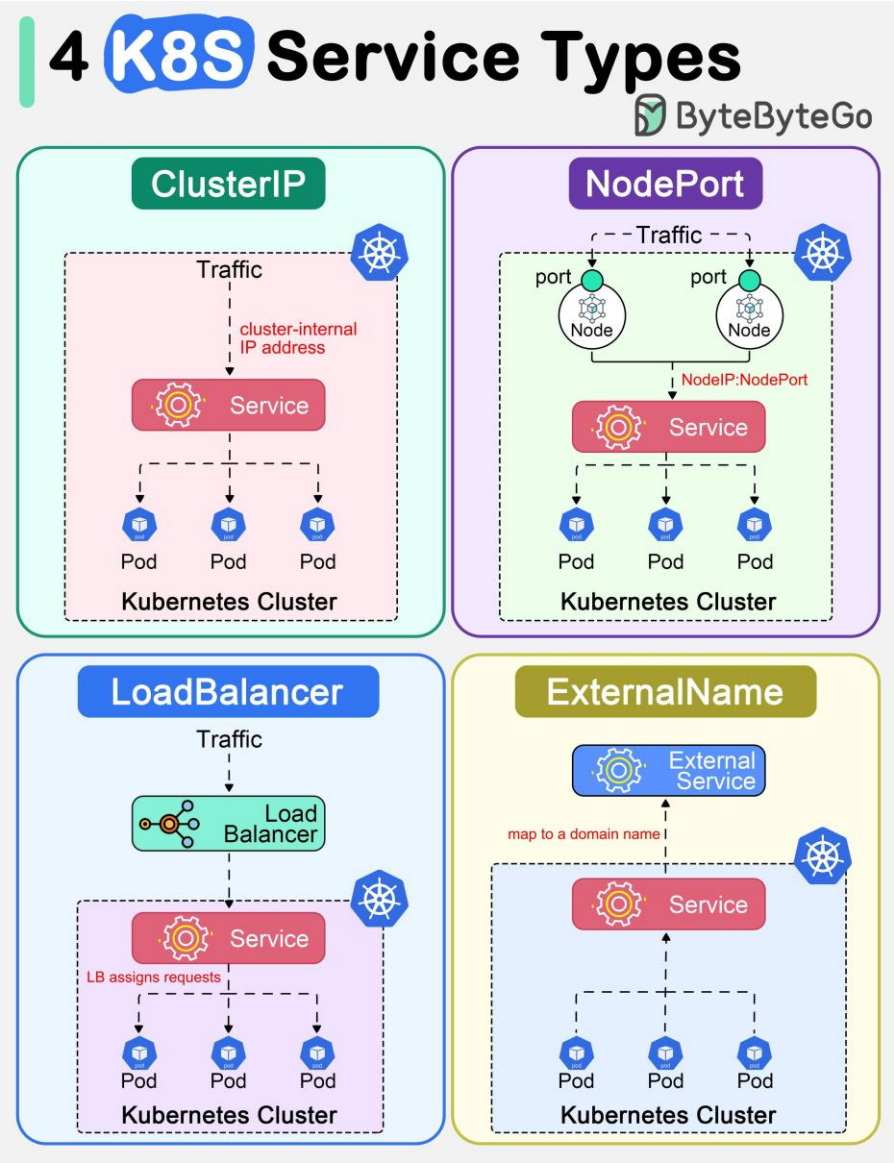
With Service:

Service IP = 10.96.0.45 —► always the same, always routes to healthy pods

It also load balances, allows DNS as well.

A pod DNS: 10-0-0-2.default.pod.cluster.local
A service DNS: nginx-svc.default.svc.cluster.local

Services



Every LB has a NodePort, every NodePort has a ClusterIP. K3d has a default LB but there are many others.

Port Range:

- ClusterIP: Any
- NodePort: 30000 – 32767
- LB: Any

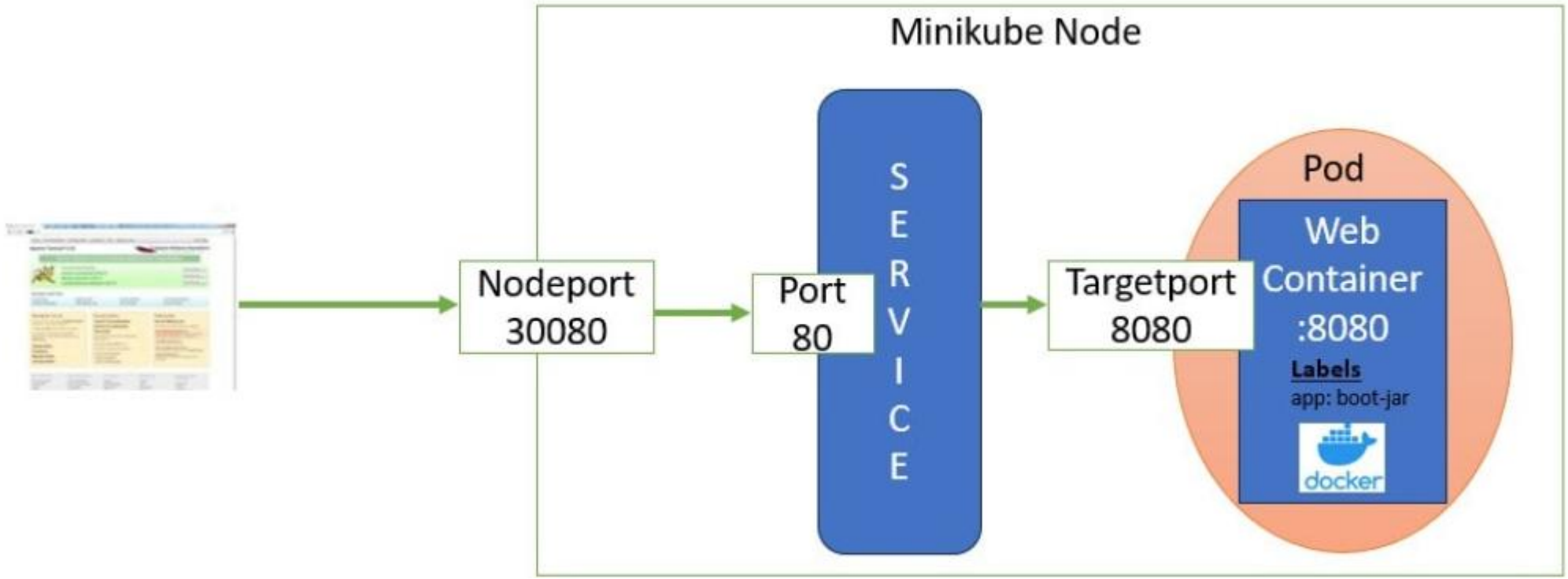
Services

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
```

Services



Source: StackOverflow

Services - DNS

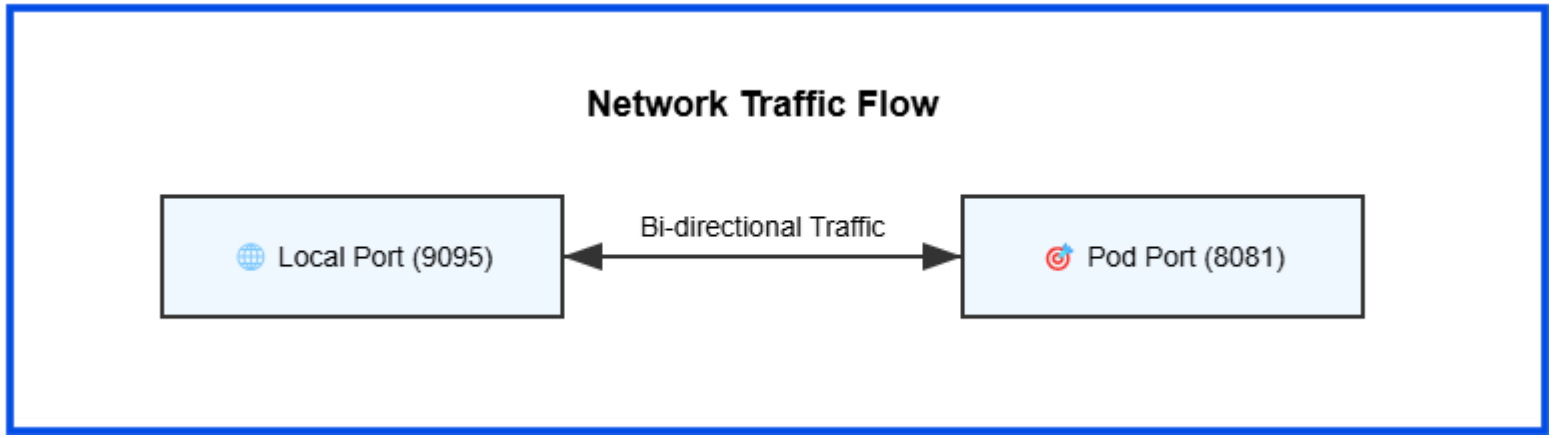
The full DNS:
<pod-ipv4-address>.<service-name>.<my-namespace>.svc.<cluster-domain.example>

Get pod IP via:
sudo kubectl get pod –o wide
sudo kubectl create –f 12B_Service_NodePort.yaml

Login into a non-service pod:
sudo kubectl exec –it curl-pod – sh

Run, for testing:
- curl <pod-IP>
- curl <pod-IP>.default.pod.cluster.local
- curl nginx-nodeport.default.svc.cluster.local

Port forward



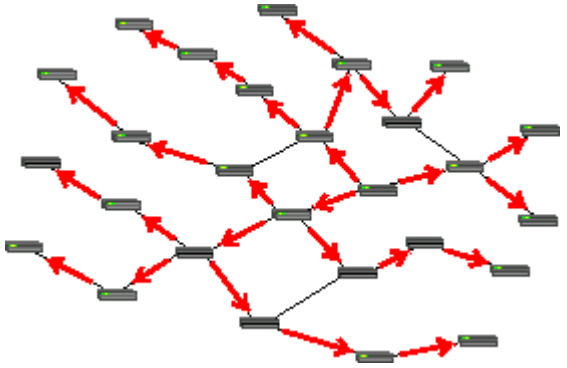
Expose the port from a kubernetes pod/service to you.
Creates a tunnel through the K8S Api server to the pod/svc

Port forward

From a service:
sudo kubectl port-forward svc/nginx-nodeport 8080:80

From the deployment (random pod):
sudo kubectl port-forward deployment/nginx 8080:80

From a pod:
sudo kubectl port-forward pod/<pod-name> 8080:80



StatefulSets

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: nginx
spec:
 serviceName: nginx-headless
 replicas: 3
 selector:
  matchLabels:
   app: nginx
 template:
  metadata:
   labels:
    app: nginx
  spec:
   containers:
    - name: nginx
      image: nginx
      volumeMounts:
       - name: data
         mountPath: /data
 volumeClaimTemplates:
  - metadata:
     name: data
    spec:
     accessModes: ["ReadWriteOnce"]
     resources:
      requests:
       storage: 1Gi
```

```yaml
apiVersion: v1
kind: Service
metadata:
 name: nginx-headless
spec:
 clusterIP: None
 selector:
  app: nginx
 ports:
  - port: 80
```

- for stateful applications
- each pod is numbered
- always keep the same name
- created/removed in sequential order
- each pod has its own PVC
- Needs to be connected with a headless service (so you can ping nginx-1, nginx-2, etc, instead of Ips that will change regardless)

- Also supports rollout/rollback ☺

Ingress

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: nginx-ingress
spec:
 ingressClassName: traefik
 rules:
  - host: nginx.local
    http:
     paths:
      - path: /
        pathType: Prefix
        backend:
         service:
          name: nginx-svc
          port:
           number: 80
```

Which ingress handles the rule

Request only matches this header

Protocol

Patch to be matched (nginx.local/) and how to match it (prefix, Exact)

Where to forward matching traffic

Other ingress for example is the nginx one:

kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml

Ingress

Create a new k3d cluster with support to Ingress points:
sudo k3d cluster create my-cluster --port "8080:80@loadbalancer"
(if you are using a managed cluster, chances are this is already done to you)

Run:
echo "127.0.0.1 nginx.local" | sudo tee -a /etc/hosts

Then you can: curl http://nginx.local:8080 outside Kubernetes!
Alternatively, curl http://localhost:8080 -H "Host: nginx.local"


DEMO
TIME

ServiceAccount

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
---
apiVersion: v1
kind: Pod
metadata:
  name: sa-pod
spec:
  serviceAccountName: my-service-account
  containers:
    - name: alpine
      image: alpine
      command: ["sleep", "3600"]
```

- An user account but for non-humans
- Have access to the Kubernetes API as well ☺

Give it permissions:
kubectl create rolebinding msa-readonly --clusterrole=view --serviceaccount=default:my-service-account --namespace=default

Login into the pod and try:
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
CACERT=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
curl -s --cacert $CACERT -H "Authorization: Bearer $TOKEN" \
https://kubernetes.default.svc/api/v1/namespaces/default/pods

You can even install kubectl and run there!

RBAC

Role-Based Access Control

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
subjects:
  - kind: User
    name: admin
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

This is for the core group API (v1)

Pods, services, nodes, configmaps

No access to: create, delete, update, patch

Matches the role to the subject:

- **ServiceAccount**
- **Users**
- **Group**

See your user (if using kind: User):

kubectl config view --raw -o jsonpath='{.users[0].user.client-certificate-data}' | \ base64 -d | \ openssl x509 -noout –subject

(Return on k3d will be CN=admin)

* You can also use a ServiceAccount, or create new users/keys for limited permissions as well.

ConfigMaps & Secrets

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_ENV: "production"
  APP_PORT: "8080"
  config.yaml: |
    server:
      port: 8080
      debug: false
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  namespace: default
type: Opaque
stringData:
  DB_PASSWORD: "supersecret"
  API_KEY: "abc123"
```

- Contains data that can be read-only mounted to a system
- Almost identical, except for encoding (plaintext vs base64)
- Secret = sensitive, ConfigMap = not sensitive
- Secret supports different ”types”

Run pod and check vars inside it (cmap):
echo $APP_ENV
echo $APP_PORT

Check mounted file (cmap):
cat /etc/config/config.yaml

Run pod and check vars inside it (skey):
echo $DB_PASSWORD
echo $API_KEY

Check mounted file (skey):
cat /etc/secrets/*

The base64 is in etcd! Restrict secrets with RBAC!

PriorityClass

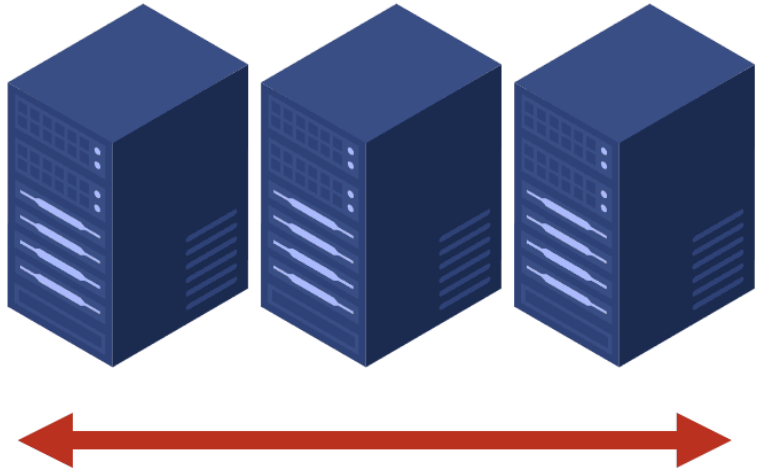
```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority-nonpreempting
value: 1000000
preemptionPolicy: Never
globalDefault: false
description: "This priority class will not cause other pods to be preempted."
```

- Ensure which pods will be scheduled first ☺
- Two default priorities in k8s: system-cluster-critical and system-node-critical

HorizontalPodAutoscaler

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: nginx-hpa
spec:
 scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: nginx
 minReplicas: 2
 maxReplicas: 10
 metrics:
  - type: Resource
   resource:
    name: cpu
    target:
     type: Utilization
     averageUtilization: 50
```

Add more resources like virtual machines to your system to spread out the workload across them.



Testing the HPA!

Start the HPA + Pod script!

Create a load generator:
kubectl run load-generator --image=busybox \ --restart=Never \ -- sh -c "while true; do wget -q -O- http://nginx-svc; done“

Get HPA count:
kubectl get hpa nginx-hpa –w

Watch CPU usage:
watch kubectl top pods

Watch pods scaling up:
kubectl get pods -w

Briefly: Custom Resource Definition (CRD)

https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/

...Maybe?


TIME FOR
A BREAK!

Jupyterhub, Prometheus and Grafana

Helm

A package manager for Kubernetes ☺

To install:

curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
chmod 700 get_helm.sh
./get_helm.sh

Or:

brew install helm

Or:

sudo apt-get install curl gpg apt-transport-https --yes
curl -fsSL https://packages.buildkite.com/helm-linux/helm-debian/gpgkey | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/helm.gpg] https://packages.buildkite.com/helm-linux/helm-debian/any/ any main" | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list
sudo apt-get update
sudo apt-get install helm

Jupyterhub

helm repo add jupyterhub https://hub.jupyter.org/helm-chart/
helm repo update

helm upgrade --cleanup-on-fail \
 --install <helm-release-name> jupyterhub/jupyterhub \
 --namespace <k8s-namespace> \
 --create-namespace \
 --version=<chart-version> \
 --values config.yaml

Port forward it:
kubectl port-forward svc/proxy-public 8888:80
(put an ingress if on an actual cluster)

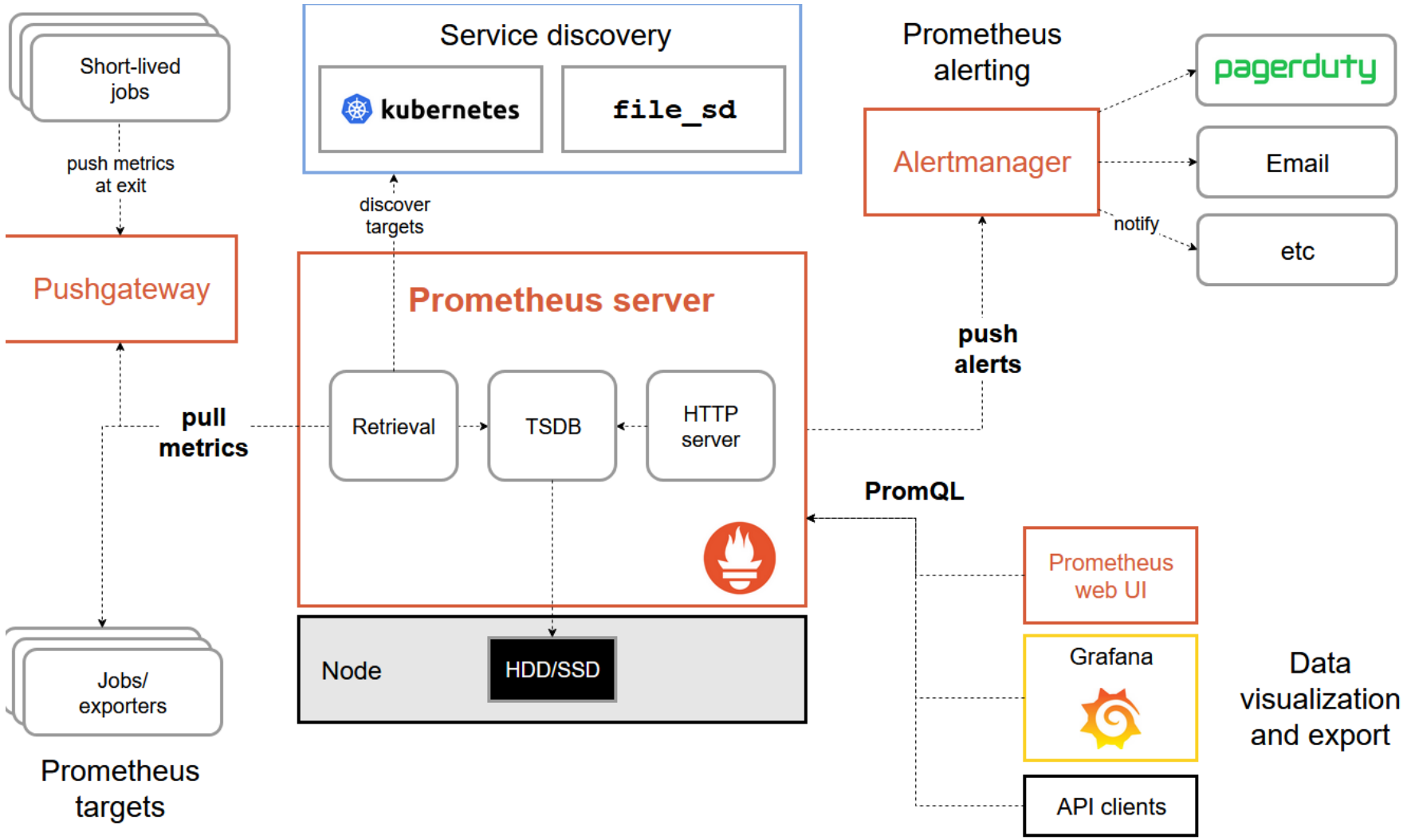
See releases using **helm list**.
To remove, **helm uninstall <release-name>**
To see values: **helm get values<release-name>**

values.yaml:

```
hub:
  config:
    DummyAuthenticator:
      password: "123456"
    JupyterHub:
      authenticator_class: dummy
```

More info:
https://z2jh.jupyter.org/en/stable/jupyterhub/customizing/extending-jupyterhub.html

Prometheus Architecture



Prometheus Data format

Normal format:
HELP <metric_name> <description>
TYPE <metric_name>
<type> <metric_name>{<label_key>="<label_value>", ...} <value> [timestamp]

Examples:
HELP http_requests_total Total number of HTTP requests
TYPE http_requests_total counter
http_requests_total{method="GET", status="200", service="api"} 1027
http_requests_total{method="POST", status="500", service="api"} 3

HELP cpu_usage_percent CPU usage percentage
TYPE cpu_usage_percent gauge
cpu_usage_percent{host="node-1"} 72.4
cpu_usage_percent{host="node-2"} 45.1

Type	Use cases
counter	Only goes up (errors, requests)
gauge	Goes up and down (CPU, memory)
histogram	Latency distribution with buckets
summary	Same as above but with quantities

Let’s deploy Prometheus & Grafana

Install through helm:
helm install <release-name> oci://ghcr.io/prometheus-community/charts/kube-prometheus-Stack

Access Prometheus UI:
kubectl port-forward svc/<release-name>-kube-prom-prometheus 9090:9090

- Look for a metric like http_requests_total or container_cpu_usage_seconds_total ☺
- Use submetric like avg()

Access Grafana UI:
kubectl --namespace default get secrets <release-name>-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo
kubectl port-forward svc/<release-name>-grafana 3000:80

Kube-state-metrics is deployed as well: https://github.com/kubernetes/kube-state-metrics
…Alternatively you can scrape directly from a kubelet too via the Kubernetes API (e.g. Python, go, etc).

To add into Prometheus:

Push yourself:
Need to install pushgateway first!
helm install <release-name> oci://ghcr.io/prometheus-community/charts/prometheus-pushgateway

Then, every time you need to push:
kubectl port-forward svc/pushgateway-prometheus-pushgateway 9091:9091
echo "my_metric 42" | curl --data-binary @- http://localhost:9091/metrics/job/test_job

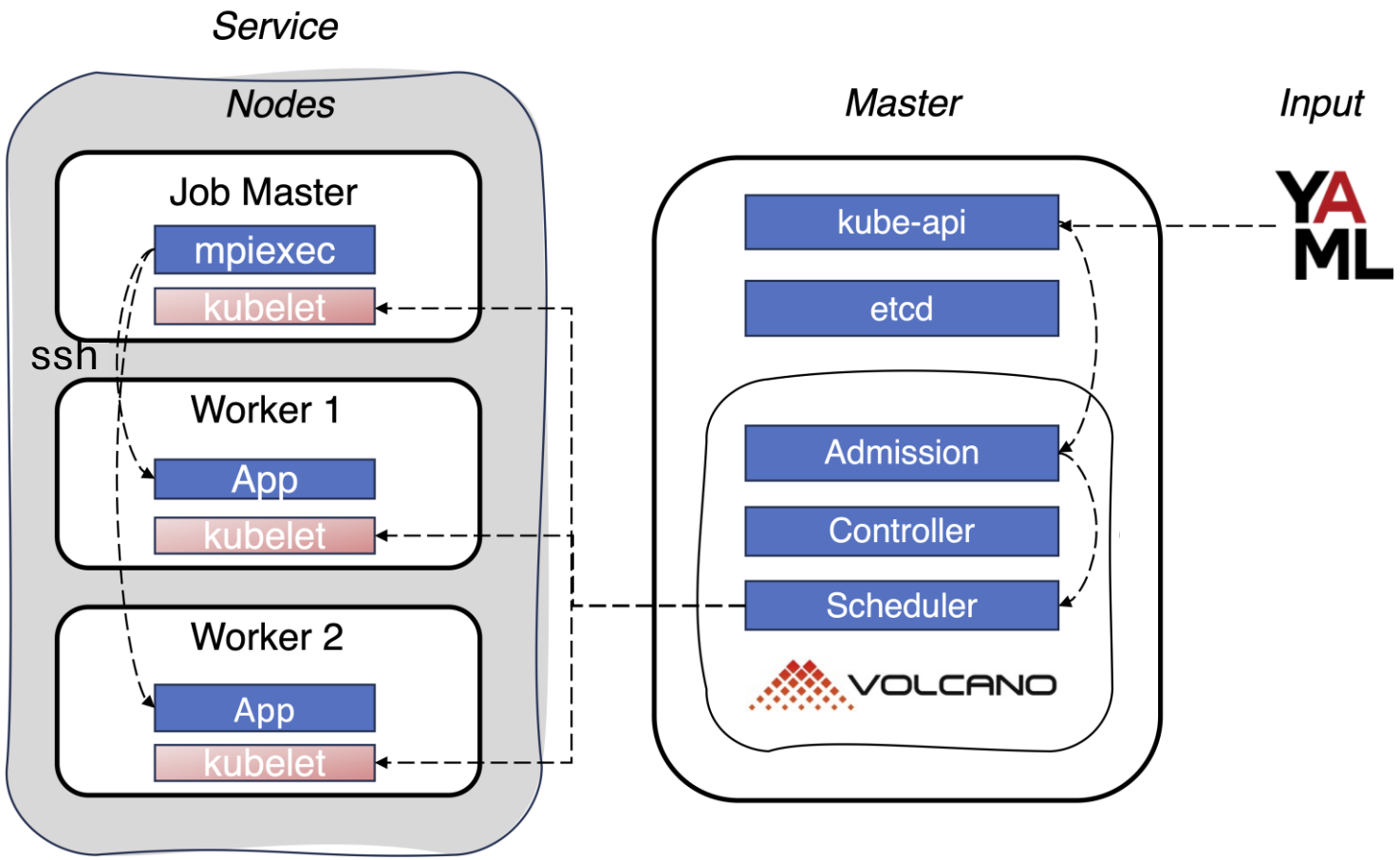
Now let Prometheus scrape it:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: pushgateway-monitor
  namespace: default
  labels:
    release: <your-release-name-for-prometheus>
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: prometheus-pushgateway
  endpoints:
    - port: http
      interval: 30s
      path: /metrics
```

Type	Use cases
counter	Only goes up (errors, requests)
gauge	Goes up and down (CPU, memory)
histogram	Latency distribution with buckets
summary	Same as above but with quantities

HPC and AI

Volcano and MPI (Distributed Applications)



Master:
mpiexec --allow-run-as-root -wdir /gromacs/ --host **MPI_HOST** -np 4 gmx_mpi mdrun -s benchMEM.tpr -ntomp 1 -cpi state.cpt

Workers:
/usr/sbin/sshd

Kubernetes YAML for jobs

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: fe-job
spec:
  template:
    spec:
      containers:
      - name: fe-job
        image: raijenki/mpik8s:minife
        command: ["/bin/bash"]
        args: ["-c", "cd
/root/miniFE/openmp/src && ./miniFE.x -nx 1000
-ny 1000 -nz 1000;"]
        imagePullPolicy: Always
      restartPolicy: OnFailure
```

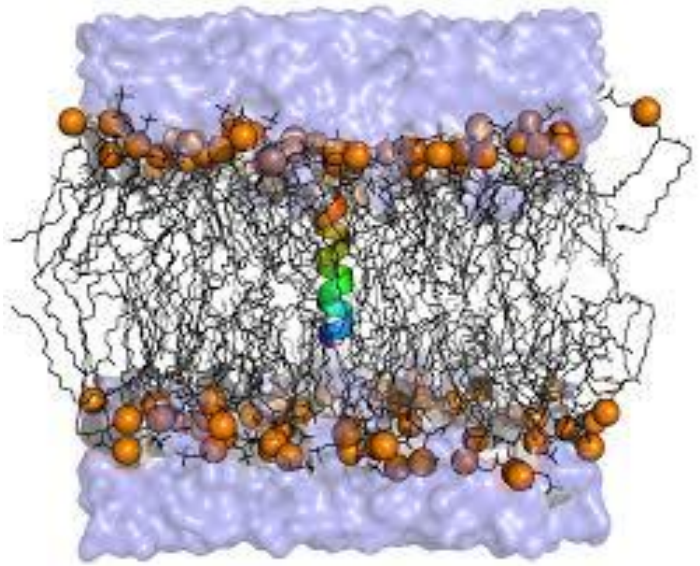
(env var for OMP and resources are omitted)

Install Volcano!

Execute helm:
helm repo add volcano-sh https://volcano-sh.github.io/helm-charts
helm install <release-name> volcano-sh/volcano -n volcano-system --create-namespace

Run a distributed application!

FAST. FLEXIBLE. FREE.
GROMACS



vLLM demonstration in a real cluster ☺

Thank you for today!

We’ll later send an e-mail to you, feedback will be really appreciated!